

CALCULUS

for Machine Learning, Deep Learning, AI & Data Science

Beginner-Friendly • Intuition-Driven • 60+ Pages

Complete coverage of derivatives, gradients, chain rule, backpropagation, optimization, Taylor series, and every calculus tool used in modern ML/AI

Author: Your Name | Date: 2025 | Version 1.0

Designed for ML students, data scientists, and AI practitioners

Table of Contents

Table of Contents	2
Introduction	5
What is Calculus?	5
Why Calculus is Fundamental to Machine Learning and AI.....	5
How Calculus Connects with Optimisation and Learning	6
Chapter 1: Functions and Graphs	7
1.1 What is a Function?.....	7
Common Functions in ML	7
Composite Functions	7
1.2 Graphs and Visualisation.....	8
1.3 Limits and Continuity	9
What is a Limit?.....	9
Continuity.....	9
The Fundamental Limit That Defines the Derivative	10
Chapter 2: Single-Variable Derivatives	11
2.1 The Derivative — Rate of Change	11
What is a Derivative?	11
Computing the Derivative from First Principles.....	11
2.2 Differentiation Rules	12
Power Rule	12
Sum Rule	12
Product Rule.....	12
Quotient Rule	12
Chain Rule — The Most Important Rule for ML.....	13
2.3 Derivatives of Common Functions	13
Deriving the Sigmoid Derivative — Step by Step	14
Chapter 3: Optimization and Critical Points.....	16
3.1 What Optimization Means in ML	16
3.2 Critical Points — Maxima, Minima, Saddle Points	16
Where Derivatives Equal Zero	16
The Second Derivative Test.....	17
3.3 Derivatives of Loss Functions.....	17
MSE Loss — Mean Squared Error	17

Binary Cross-Entropy (Log-Loss).....	17
Chapter 4: The Gradient	19
4.1 Moving to Multiple Variables.....	19
4.2 Partial Derivatives	19
What is a Partial Derivative?	19
Step-by-Step: Partial Derivatives of the MSE Loss	19
4.3 The Gradient Vector.....	20
Assembling Partial Derivatives into the Gradient	20
Chapter 5: Gradient Descent	22
5.1 The Core Algorithm	22
Gradient Descent in Plain English	22
5.2 The Learning Rate — α	22
5.3 Variants of Gradient Descent.....	23
5.4 Gradient Descent for Linear Regression — Full Example	23
Chapter 6: Chain Rule and Backpropagation	25
6.1 The Chain Rule — Derivatives of Composite Functions.....	25
Single-Variable Chain Rule	25
Extended Chain Rule for Longer Chains.....	25
Multivariable Chain Rule.....	25
6.2 Backpropagation — Chain Rule in Neural Networks	25
What is Backpropagation?	25
Worked Example: 2-Layer Network	26
The Vanishing Gradient Problem	27
Chapter 7: Jacobian and Hessian Matrices	29
7.1 The Jacobian Matrix.....	29
From Vectors to Vectors	29
Worked Example: Jacobian of a 2D Function	29
7.2 The Hessian Matrix	30
Second-Order Derivatives — Curvature.....	30
Example: Hessian of $f(x,y) = x^2 + xy + y^2$	30
Newton's Method — Using the Hessian	30
Chapter 8: Advanced Optimisation Techniques	32
8.1 Momentum	32
Why Standard GD Oscillates	32
8.2 RMSprop and Adam	32

8.3 Learning Rate Schedules	33
Chapter 9: Neural Networks — The Full Calculus	34
9.1 A Neural Network is a Composite Function	34
9.2 Full Backpropagation Algorithm	34
9.3 Batch Operations — Matrix Form of Backprop.....	35
Chapter 10: Logistic Regression — Calculus in Action	37
10.1 The Model	37
10.2 Binary Cross-Entropy Loss.....	37
10.3 Gradient Derivation — Step by Step.....	37
Chapter 11: Higher-Order Derivatives and Taylor Series.....	39
11.1 Higher-Order Derivatives	39
Sign of Second Derivative → Convexity	39
11.2 Taylor Series — Approximating Functions Locally	39
What is a Taylor Series?	39
Important Taylor Expansions	40
Chapter 12: Numerical Differentiation and Automatic Differentiation.....	42
12.1 Numerical Differentiation	42
Finite Differences	42
12.2 Automatic Differentiation (Autograd)	42
What is Autograd?	42
Chapter 13: Calculus in Probability and Statistics.....	45
13.1 Probability Density Functions	45
13.2 Maximum Likelihood Estimation	45
13.3 Gradient of the KL Divergence	46
Chapter 14: Summary and Further Reading	47
14.1 Complete Concept Map	47
14.2 The Big Picture in 5 Steps.....	47
14.3 Recommended Next Steps.....	48
Mathematics	48
Books.....	48
Practical Projects.....	48
14.4 Final Comprehensive Quiz	48

Introduction

What is Calculus?

Calculus is the mathematics of change. Where ordinary algebra handles static quantities — 'what is 3 plus 4?' — calculus asks dynamic questions: 'How fast is this value changing?', 'Where does this function reach its smallest value?', and 'How do small changes in one variable ripple through a chain of calculations?'

Calculus was developed independently by Isaac Newton and Gottfried Wilhelm Leibniz in the 17th century. Newton needed it to describe the motion of planets. Today, machine learning engineers need it to train neural networks. The problems are different; the mathematics is the same.

Two Pillars of Calculus

Differential Calculus — about RATES OF CHANGE.

- How fast is the loss function decreasing?
- Which direction should we move the weights to reduce error?

Integral Calculus — about ACCUMULATION.

- Area under a probability density curve
- Summing up infinite tiny contributions

This book focuses heavily on differential calculus, as it is the engine of machine learning.

Why Calculus is Fundamental to Machine Learning and AI

Every time a neural network trains on data, calculus is running in the background. The training process is fundamentally an optimisation problem: find the model parameters (weights) that minimise a loss function. To find the minimum, you need to know which direction is 'downhill' — and that requires derivatives.

Calculus Concept	Where It Appears in ML/AI
Derivatives	Gradient descent updates: how to change weights to reduce loss
Partial Derivatives	Gradients of loss w.r.t. each individual weight
Chain Rule	Backpropagation in neural networks
Jacobian Matrix	Gradients in vector-valued layers (batch norm, softmax)

Hessian Matrix	Second-order optimisation, Newton's method, curvature
Taylor Series	Understanding optimisation algorithms, approximations
Gradient Descent	The core training algorithm for nearly all ML models
Numerical Differentiation	Gradient checking, automatic differentiation

How Calculus Connects with Optimisation and Learning

A machine learning model learns by adjusting its parameters to minimize a loss function — a number measuring how wrong its predictions are. Calculus tells us the exact direction to move each parameter to make the loss smaller.

Real-World Analogy

Imagine you are blindfolded on a hilly landscape and want to find the lowest valley. You cannot see the whole landscape, but you can feel which direction is downhill under your feet right now. You take a small step in that direction, feel the slope again, take another step. That feeling of 'which direction is downhill' is the gradient — and the process of following it is gradient descent.

Key Insight

You do not need to be a calculus expert before starting ML. But understanding the key ideas — especially derivatives, the chain rule, and gradients — will let you truly understand what your model is doing during training, rather than treating it as a black box.

Chapter 1: Functions and Graphs

1.1 What is a Function?

A function is a rule that takes an input and produces exactly one output. We write $f(x)$ = some formula, where x is the input and $f(x)$ is the output.

$$f: X \rightarrow Y \quad (\text{function } f \text{ maps from set } X \text{ to set } Y)$$

Notation: $y = f(x)$ means 'y is the output when x is the input'

Example: $f(x) = x^2 \quad f(3) = 9, \quad f(-2) = 4$
Example: $g(x) = 2x + 1 \quad g(0) = 1, \quad g(5) = 11$

Common Functions in ML

Function	Formula	ML Use
Linear	$f(x) = wx + b$	Regression, linear layers
Quadratic	$f(x) = x^2$	MSE loss (squared error)
Exponential	$f(x) = e^x$	Softmax, probability distributions
Logarithm	$f(x) = \ln(x)$	Log-loss, entropy, KL divergence
Sigmoid	$f(x) = 1/(1+e^{-x})$	Binary classification output
ReLU	$f(x) = \max(0, x)$	Hidden layer activations
Tanh	$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	Recurrent networks (LSTM/GRU)

Composite Functions

A composite function applies one function to the output of another. This is written as $f(g(x))$ or $(f \circ g)(x)$. Composite functions are absolutely central to deep learning — a neural network is a long chain of composed functions.

$$\text{If } g(x) = 2x \text{ and } f(u) = u^2$$
$$\text{then } (f \circ g)(x) = f(g(x)) = f(2x) = (2x)^2 = 4x^2$$

 **Neural Networks as Composite Functions**

A 3-layer neural network computes:

$\text{output} = f_3(f_2(f_1(x)))$

Where each f_i is: $\text{activation}(W_i \times \text{input} + b_i)$

Training this network requires differentiating through all these composed functions — which is exactly what the chain rule does.

1.2 Graphs and Visualisation

The graph of a function $f(x)$ is the set of all points $(x, f(x))$ plotted on a 2D plane. The x-axis shows inputs; the y-axis shows outputs. Graphs make abstract functions concrete and intuitive.

Key features to observe on a graph:

- Where is the function increasing? (slope points upward)
- Where is it decreasing? (slope points downward)
- Where does it reach a maximum or minimum? (slope = 0)
- Is it smooth (differentiable) everywhere?
- Does it have any jumps or breaks (discontinuities)?

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 400)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.plot(x, x**2, 'b-', linewidth=2)
plt.title('f(x) = x2 (Quadratic)')
plt.xlabel('x'); plt.ylabel('f(x)'); plt.grid(True)

plt.subplot(1, 3, 2)
plt.plot(x, np.exp(x), 'r-', linewidth=2)
plt.title('f(x) = ex (Exponential)')
plt.xlabel('x'); plt.ylabel('f(x)'); plt.grid(True)

plt.subplot(1, 3, 3)
plt.plot(x, 1/(1+np.exp(-x)), 'g-', linewidth=2)
plt.title('f(x) = σ(x) (Sigmoid)')
plt.xlabel('x'); plt.ylabel('f(x)'); plt.grid(True)

plt.tight_layout()
plt.show()
```


 Mini Quiz**Q1: What is $f(4)$ if $f(x) = 3x^2 - 2x + 1$?** **Answer:** $f(4) = 3(16) - 2(4) + 1 = 48 - 8 + 1 = 41$ **Q2: Why is understanding composite functions important for deep learning?** **Answer:** A neural network IS a composite function — each layer's output feeds as input to the next layer. The chain rule, which differentiates composite functions, is the mathematics behind backpropagation.

1.3 Limits and Continuity

What is a Limit?

A limit asks: 'What value does $f(x)$ approach as x gets closer and closer to some number a ?' We do not ask what $f(a)$ equals — we ask what $f(x)$ is heading toward.

$$\lim_{x \rightarrow a} f(x) = L \quad \text{means: as } x \rightarrow a, f(x) \rightarrow L$$

Example: $\lim_{x \rightarrow 2} (x^2) = 4$ (as x approaches 2, x^2 approaches 4)

 Real-World Analogy

Imagine you are walking along a road toward a bridge. A limit is asking: 'Where are you heading?' — not 'Are you on the bridge yet?' You might never reach the bridge (maybe there's a fence), but you can still know where you're heading. The limit is your destination.

Continuity

A function is continuous at a point $x = a$ if three conditions hold: the function is defined at a , the limit exists at a , and the limit equals the function value. In plain English: you can draw the function without lifting your pencil.

Why Continuity Matters in ML

Smooth (continuous and differentiable) functions are essential for gradient descent.

- Sigmoid, tanh: smooth everywhere $\rightarrow$ easy to differentiate

- ReLU: continuous but NOT differentiable at $x=0$ (has a sharp corner)
→ In practice, we just define the derivative at 0 to be 0 or 1 and it works fine

Functions that are discontinuous (have jumps) would cause gradient descent to fail catastrophically — the gradient would be undefined or infinite at the jump.

The Fundamental Limit That Defines the Derivative

The derivative is defined as a limit. This one limit formula is the foundation of all differential calculus:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

This says: the derivative is the limit of the slope of a secant line as it gets infinitely close to a tangent line.

Chapter 2: Single-Variable Derivatives

2.1 The Derivative — Rate of Change

What is a Derivative?

The derivative of a function at a point tells you the instantaneous rate of change — the slope of the function at that exact point. It answers: 'If I increase x by a tiny amount, how much does $f(x)$ change?'

$$f'(x) = df/dx = \text{rate of change of } f \text{ with respect to } x$$

Geometric meaning: slope of the tangent line to $f(x)$ at point x

If $f'(x) > 0$: function is increasing at x

If $f'(x) < 0$: function is decreasing at x

If $f'(x) = 0$: function has a horizontal tangent (potential maximum or minimum)



Real-World Analogy

Imagine you are driving a car. Your position as a function of time is $f(t)$. Your speed (speedometer reading) at any instant is the derivative $f'(t)$ — the rate of change of position. If $f'(t) = 60$, you are moving at 60 km/h at that instant. If $f'(t) = 0$, you are stopped. If $f'(t) < 0$, you are reversing.

Computing the Derivative from First Principles

Let us compute the derivative of $f(x) = x^2$ from the definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} = \lim_{h \rightarrow 0} (2x + h) = 2x \end{aligned}$$

Therefore: $d/dx (x^2) = 2x$

💡 Key Insight

The derivative of x^2 at $x=3$ is $2 \times 3 = 6$. This means: at the point $x=3$, for every unit increase in x , $f(x) = x^2$ increases by approximately 6 units. The tangent line has slope 6.

2.2 Differentiation Rules

Computing derivatives from the definition every time would be tedious. Instead, we use rules that work for common function types.

Power Rule

$$d/dx (x^n) = n \cdot x^{n-1}$$

Examples:

$$d/dx (x^3) = 3x^2$$

$$d/dx (x^5) = 5x^4$$

$$d/dx (x) = 1 \quad (n=1)$$

$$d/dx (1) = 0 \quad (\text{constant: } n=0)$$

$$d/dx (\sqrt{x}) = 1/(2\sqrt{x}) \quad (n=1/2)$$

$$d/dx (1/x) = -1/x^2 \quad (n=-1)$$

Sum Rule

$$d/dx [f(x) + g(x)] = f'(x) + g'(x)$$

$$\text{Example: } d/dx (x^3 + 5x^2 - 3x + 7) = 3x^2 + 10x - 3$$

Product Rule

$$d/dx [f(x) \cdot g(x)] = f'(x) \cdot g(x) + f(x) \cdot g'(x)$$

$$\text{Example: } d/dx [x^2 \cdot \sin(x)] = 2x \cdot \sin(x) + x^2 \cdot \cos(x)$$

Memory trick: 'first times derivative of second PLUS second times derivative of first'

Quotient Rule

$$d/dx [f(x)/g(x)] = [f'(x) \cdot g(x) - f(x) \cdot g'(x)] / [g(x)]^2$$

$$\text{Example: } d/dx [x^2/(x+1)] = [2x(x+1) - x^2 \cdot 1] / (x+1)^2 = (x^2 + 2x)/(x+1)^2$$

Memory trick: 'low d-high minus high d-low, over low squared'

Chain Rule — The Most Important Rule for ML

The chain rule handles derivatives of composite functions. If $y = f(g(x))$, then:

$$dy/dx = dy/du \cdot du/dx \quad \text{where } u = g(x)$$

$$\text{Or: } d/dx f(g(x)) = f'(g(x)) \cdot g'(x)$$

$$\text{Example: } d/dx [(2x+1)^4]$$

$$\text{Let } u = 2x+1, \text{ so } f(u) = u^4$$

$$dy/du = 4u^3 = 4(2x+1)^3$$

$$du/dx = 2$$

$$\text{Answer: } 4(2x+1)^3 \times 2 = 8(2x+1)^3$$

Chain Rule = Backpropagation

The chain rule is the mathematical foundation of backpropagation — the algorithm used to train every neural network.

A neural network is a composition of many functions: $\text{output} = f_3(f_2(f_1(x)))$

To train it, we need $\partial \text{Loss} / \partial w$ for each weight w . The chain rule lets us compute this by multiplying the derivatives layer by layer — backward through the network.

Without the chain rule, training neural networks would be mathematically impossible.

2.3 Derivatives of Common Functions

Function $f(x)$	Derivative $f'(x)$	Notes
x^n	$n \cdot x^{n-1}$	Power rule
e^x	e^x	e^x is its own derivative!

a^x	$a^x \cdot \ln(a)$	General exponential
$\ln(x)$	$1/x$	Natural log
$\log_a(x)$	$1/(x \cdot \ln(a))$	Log base a
$\sin(x)$	$\cos(x)$	Trig derivative
$\cos(x)$	$-\sin(x)$	Trig derivative
$\tan(x)$	$1/\cos^2(x) = \sec^2(x)$	Trig derivative
$\sigma(x) = 1/(1+e^{-x})$	$\sigma(x) \cdot (1 - \sigma(x))$	Sigmoid — elegant!
$\tanh(x)$	$1 - \tanh^2(x)$	Tanh
ReLU: $\max(0, x)$	0 if $x < 0$, 1 if $x > 0$	Piecewise (undefined at 0)

Deriving the Sigmoid Derivative — Step by Step

The sigmoid function $\sigma(x) = 1/(1+e^{-x})$ appears everywhere in ML. Let us derive its derivative:

$$\sigma(x) = 1/(1+e^{-x}) = (1+e^{-x})^{-1}$$

$$\text{Using chain rule: } d/dx [(1+e^{-x})^{-1}] = -(1+e^{-x})^{-2} \cdot (-e^{-x})$$

$$= e^{-x} / (1+e^{-x})^2$$

$$= [1/(1+e^{-x})] \cdot [e^{-x}/(1+e^{-x})]$$

$$= \sigma(x) \cdot (1 - \sigma(x)) \quad \leftarrow \text{elegant!}$$

Key Insight

The sigmoid derivative $\sigma(x) \cdot (1 - \sigma(x))$ is the reason sigmoid neurons 'saturate' during training. When $\sigma(x)$ is near 0 or 1 (which happens when $|x|$ is large), this derivative becomes nearly zero. Gradients stop flowing. This is the 'vanishing gradient problem', and it motivated the invention of ReLU.

```
import numpy as np
import matplotlib.pyplot as plt

# Activation functions and their derivatives
x = np.linspace(-5, 5, 400)

def sigmoid(x):      return 1 / (1 + np.exp(-x))
def sigmoid_deriv(x): return sigmoid(x) * (1 - sigmoid(x))
def relu(x):         return np.maximum(0, x)
def relu_deriv(x):   return (x > 0).astype(float)
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

axes[0,0].plot(x, sigmoid(x), 'b-', lw=2); axes[0,0].set_title('Sigmoid')
axes[0,1].plot(x, sigmoid_deriv(x), 'r-', lw=2); axes[0,1].set_title('Sigmoid
Derivative')
axes[1,0].plot(x, relu(x), 'g-', lw=2); axes[1,0].set_title('ReLU')
axes[1,1].plot(x, relu_deriv(x), 'm-', lw=2); axes[1,1].set_title('ReLU
Derivative')

for ax in axes.flat: ax.grid(True); ax.axhline(0, color='k', lw=0.5)
plt.tight_layout(); plt.show()
```

Mini Quiz

Q1: What is $d/dx (3x^4 - 7x^2 + 2x - 5)$?

✓ **Answer:** $12x^3 - 14x + 2$ (apply power rule term by term)

Q2: What is $d/dx [e^x \cdot \ln(x)]$?

✓ **Answer:** Using product rule: $e^x \cdot \ln(x) + e^x \cdot (1/x) = e^x(\ln(x) + 1/x)$

Q3: Why does ReLU help prevent the vanishing gradient problem compared to Sigmoid?

✓ **Answer:** ReLU's derivative is 1 for all positive inputs (not squeezed toward zero). So gradients flow without shrinking for positive activations. Sigmoid's derivative goes to 0 for large $|x|$, killing gradients.

Chapter 3: Optimization and Critical Points

3.1 What Optimization Means in ML

Every machine learning algorithm is — at its heart — an optimization problem. We define a loss function $L(w)$ that measures how wrong our model is, and we want to find the weights w that make $L(w)$ as small as possible.

$$\text{Goal: } w^* = \underset{w}{\operatorname{argmin}} L(w)$$

Where $L(w)$ could be:

$$\text{MSE: } L = (1/n) \sum (y_i - \hat{y}_i)^2$$

$$\text{Log-Loss: } L = -(1/n) \sum [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

3.2 Critical Points — Maxima, Minima, Saddle Points

Where Derivatives Equal Zero

A critical point is any location where $f'(x) = 0$ or $f'(x)$ is undefined. These are the candidates for maxima and minima. The derivative is zero because the tangent line is horizontal — the function is momentarily neither going up nor down.

Point Type	$f'(x) = 0$?	$f''(x)$	Meaning
Local Minimum	Yes	$f''(x) > 0$ (concave up)	Bowl shape — gradient descent seeks this
Local Maximum	Yes	$f''(x) < 0$ (concave down)	Hill shape — unstable for descent
Saddle Point	Yes	$f''(x) = 0$ (inconclusive)	Flat inflection — tricky for optimizers
Global Minimum	Yes	Smallest $L(w)$ everywhere	The perfect answer — often hard to find



Saddle Points in Deep Learning

In high-dimensional spaces (like a neural network with millions of weights), true local minima are rare. Most critical points are SADDLE POINTS — they are minima in some directions but maxima in others.

Surprisingly, this is OK! Research shows that in deep networks, saddle points and local minima tend to have similar loss values to the global minimum. Gradient descent usually finds a 'good enough' solution.

The Second Derivative Test

Once you find a critical point x_0 where $f'(x_0) = 0$, use the second derivative to classify it:

$$\begin{aligned} f''(x_0) > 0 &\rightarrow \text{Local minimum (concave up, like a bowl)} \\ f''(x_0) < 0 &\rightarrow \text{Local maximum (concave down, like a hill)} \\ f''(x_0) = 0 &\rightarrow \text{Test inconclusive (need more analysis)} \end{aligned}$$

Full worked example: Find and classify critical points of $f(x) = x^3 - 3x$

$$\text{Step 1: } f'(x) = 3x^2 - 3 = 3(x^2 - 1) = 0 \rightarrow x = \pm 1$$

$$\text{Step 2: } f''(x) = 6x$$

$$\text{At } x = -1: f''(-1) = -6 < 0 \rightarrow \text{LOCAL MAXIMUM } f(-1) = 2$$

$$\text{At } x = +1: f''(+1) = +6 > 0 \rightarrow \text{LOCAL MINIMUM } f(+1) = -2$$

3.3 Derivatives of Loss Functions

MSE Loss — Mean Squared Error

MSE is the standard loss for regression. For a single parameter w :

$$L(w) = (1/n) \sum_i (y_i - w \cdot x_i)^2$$

$$dL/dw = (1/n) \sum_i 2(y_i - w \cdot x_i) \cdot (-x_i)$$

$$= -(2/n) \sum_i x_i (y_i - w \cdot x_i)$$

$$\text{Gradient descent update: } w \leftarrow w - \alpha \cdot dL/dw$$

Binary Cross-Entropy (Log-Loss)

Used in logistic regression and binary classification. Here $\hat{y} = \sigma(wx+b)$ is the sigmoid output:

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

$$dL/dw = (\hat{y} - y) \cdot x \quad \leftarrow \text{beautifully clean!}$$

The gradient is just the prediction error times the input.

 Key Insight

The clean gradient of cross-entropy ($\hat{y} - y$) is one reason it is preferred over MSE for classification. When the model is very wrong ($\hat{y}$ far from y), the gradient is large and training is fast. When the model is correct ($\hat{y} \approx y$), the gradient is near zero and updates stop.

 Mini Quiz

Q1: $f(x) = x^4 - 4x^2$ has $f'(x) = 4x^3 - 8x$. Find the critical points.

✓ *Answer: $4x^3 - 8x = 4x(x^2 - 2) = 0 \rightarrow x = 0, x = \pm\sqrt{2}$ (three critical points)*

Q2: Why do we minimise loss rather than maximise it?

✓ *Answer: Loss measures 'wrongness' — how far predictions are from truth. We want predictions as close to truth as possible, meaning lowest possible error. Minimising loss = maximising model accuracy.*

Chapter 4: The Gradient

4.1 Moving to Multiple Variables

In real ML, a model has thousands or millions of parameters (weights). The loss function L depends on all of them simultaneously: $L(w_1, w_2, \dots, w_n)$. To optimise this, we need to extend the concept of a derivative to multiple variables.

4.2 Partial Derivatives

What is a Partial Derivative?

A partial derivative measures how a function changes when you vary ONE variable while holding all others constant. It is written $\partial f / \partial x$ (using ∂ instead of d).

Given $f(x, y)$, the partial derivatives are:

$\partial f / \partial x$ = derivative of f treating y as a constant

$\partial f / \partial y$ = derivative of f treating x as a constant

Example: $f(x, y) = x^2 + 3xy + y^3$

$\partial f / \partial x = 2x + 3y$ (treat y as constant, differentiate w.r.t. x)

$\partial f / \partial y = 3x + 3y^2$ (treat x as constant, differentiate w.r.t. y)

Real-World Analogy

Imagine you are on a mountain. The height $h(x, y)$ depends on your east-west position x and north-south position y . $\partial h / \partial x$ tells you the steepness if you walk EAST. $\partial h / \partial y$ tells you the steepness if you walk NORTH. Each partial derivative isolates one direction of movement.

Step-by-Step: Partial Derivatives of the MSE Loss

Consider the MSE loss with two parameters w and b : $L(w, b) = (1/n) \sum (y - wx - b)^2$

$$\partial L / \partial w = (1/n) \sum 2(y - wx - b) \cdot (-x) = -(2/n) \sum x(y - wx - b)$$

$$\partial L / \partial b = (1/n) \sum 2(y - wx - b) \cdot (-1) = -(2/n) \sum (y - wx - b)$$

These two partial derivatives tell us: 'How does the loss change if I nudge w slightly?' and 'How does it change if I nudge b slightly?'

4.3 The Gradient Vector

Assembling Partial Derivatives into the Gradient

The gradient is the vector that collects ALL partial derivatives of a function. It points in the direction of steepest ASCENT. The negative gradient points in the direction of steepest DESCENT — which is the direction we want to move to minimize the loss.

$$\nabla f(x_1, x_2, \dots, x_n) = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$$

$$\text{Example: } f(x,y) = x^2 + y^2$$

$$\nabla f = [2x, 2y]$$

$$\text{At point (1,2): } \nabla f(1,2) = [2, 4]$$

→ Steepest ascent is in direction $[2,4]$

→ Steepest descent is in direction $[-2,-4]$

The Gradient in Neural Network Training

For a neural network with n weights $[w_1, w_2, \dots, w_n]$, the gradient is:

$$\nabla L = [\partial L / \partial w_1, \partial L / \partial w_2, \dots, \partial L / \partial w_n]$$

Each element tells us: 'If I increase this weight slightly, does the loss go up or down, and by how much?'

Gradient descent updates ALL weights simultaneously:

$$w \leftarrow w - \alpha \cdot \nabla L$$

For a network with 1 million weights, this is a vector operation — fast and parallelisable on GPUs.

```
import numpy as np

# Partial derivatives: manual computation
def f(x, y):      return x**2 + 3*x*y + y**3
```

```
def df_dx(x, y): return 2*x + 3*y
def df_dy(x, y): return 3*x + 3*y**2

x, y = 2.0, 1.0
print(f'f({x},{y}) = {f(x,y)}')
print(f'∂f/∂x = {df_dx(x,y)}')
print(f'∂f/∂y = {df_dy(x,y)}')
print(f'Gradient ∇f = [{df_dx(x,y)}, {df_dy(x,y)}]')

# Numerical partial derivative (finite differences)
h = 1e-5
num_df_dx = (f(x+h, y) - f(x-h, y)) / (2*h)
num_df_dy = (f(x, y+h) - f(x, y-h)) / (2*h)
print(f'Numerical ∂f/∂x = {num_df_dx:.6f}')
print(f'Numerical ∂f/∂y = {num_df_dy:.6f}')
```

Mini Quiz

Q1: For $f(x,y) = 3x^2y + y^2$, find $\partial f/\partial x$ and $\partial f/\partial y$.

✓ Answer: $\partial f/\partial x = 6xy$ (treat y as constant) | $\partial f/\partial y = 3x^2 + 2y$ (treat x as constant)

Q2: The gradient $\nabla L = [4, -3, 1]$ at the current weights. Which weight should increase to increase the loss?

✓ Answer: w_1 , because its partial derivative $\partial L/\partial w_1 = 4 > 0$. Increasing w_1 increases L . For gradient DESCENT we would decrease w_1 .

Chapter 5: Gradient Descent

5.1 The Core Algorithm

Gradient Descent in Plain English

Gradient descent is a simple but powerful iterative algorithm for finding the minimum of a function. You start at some position (initial weights), compute the gradient (which direction is uphill), take a small step in the opposite direction (downhill), and repeat. Eventually you reach a minimum.

Repeat until convergence:

$$w \leftarrow w - \alpha \cdot \nabla L(w)$$

Where:

w = current weights (parameters)

α = learning rate (step size, small positive number, e.g. 0.01)

$\nabla L(w)$ = gradient of loss with respect to weights

$\leftarrow$ = update / assignment

5.2 The Learning Rate — α

The learning rate α is one of the most important hyperparameters in machine learning. It controls how big each step is during gradient descent.

Learning Rate	Effect	Risk
Too large ($\alpha = 1.0$)	Big steps, fast initially	Overshoots minimum, oscillates or diverges
Too small ($\alpha = 0.0001$)	Very stable, precise	Extremely slow convergence, may get stuck
Just right ($\alpha = 0.01$)	Steady, reliable descent	May still miss sharp minima
Adaptive (Adam, etc.)	Auto-adjusts per parameter	More complex, computationally heavier



Real-World Analogy

Think of rolling a marble down a bowl. The learning rate controls how hard you push the marble each time. Push too hard: the marble flies past the bottom and up the other side. Push too gently: the marble barely moves. The right push: the marble smoothly rolls to the centre.

5.3 Variants of Gradient Descent

Variant	Uses Per Step	Pros	Cons
Batch GD	All n samples	Accurate gradient, stable	Very slow for large n
Stochastic GD	1 random sample	Fast, escapes local minima	Noisy, unstable path
Mini-Batch GD	Batch of ~32-256	Best of both worlds	Batch size is another hyperparameter

5.4 Gradient Descent for Linear Regression — Full Example

Let us implement gradient descent from scratch for simple linear regression: $\hat{y} = wx + b$

```
import numpy as np
import matplotlib.pyplot as plt

# Generate synthetic data: y = 2x + 1 + noise
np.random.seed(42)
n = 100
x = np.random.randn(n)
y = 2*x + 1 + 0.5*np.random.randn(n)

# Initialise parameters
w, b = 0.0, 0.0
alpha = 0.05 # learning rate
n_steps = 200
loss_history = []

for step in range(n_steps):
    # Forward pass: compute predictions
    y_hat = w*x + b

    # Compute MSE loss
    loss = np.mean((y - y_hat)**2)
    loss_history.append(loss)

    # Compute gradients (partial derivatives of loss w.r.t. w and b)
    dL_dw = -(2/n) * np.sum(x * (y - y_hat))
    dL_db = -(2/n) * np.sum(y - y_hat)

    # Gradient descent update
    w -= alpha * dL_dw
    b -= alpha * dL_db

    if step % 40 == 0:
        print(f'Step {step:3d} | Loss: {loss:.4f} | w={w:.3f} | b={b:.3f}')

print(f'\nFinal: w={w:.3f} (true: 2.0), b={b:.3f} (true: 1.0)')

# Plot learning curve
plt.plot(loss_history)
plt.xlabel('Training Step'); plt.ylabel('MSE Loss')
plt.title('Loss Curve: Gradient Descent for Linear Regression')
```

```
plt.grid(True); plt.show()
```

Reading a Loss Curve

A healthy training loss curve:

- Starts high (model is random at first)
- Decreases steadily with each step
- Eventually flattens (converged)

Unhealthy signs:

- Loss oscillates up and down → learning rate too large
- Loss barely decreases → learning rate too small, or gradient is vanishing
- Loss increases after initial decrease → overfitting (training too long)

Mini Quiz

Q1: In gradient descent $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla L$, why do we SUBTRACT the gradient?

✓ *Answer: The gradient ∇L points in the direction of STEEPEST INCREASE of loss. To DECREASE loss, we move in the opposite direction: subtract the gradient.*

Q2: If $\alpha = 10$ and loss bounces up and down during training, what should you do?

✓ *Answer: Reduce the learning rate. A large learning rate causes overshoot — each step jumps past the minimum to the other side. Try $\alpha = 0.1$ or 0.01 .*

Chapter 6: Chain Rule and Backpropagation

6.1 The Chain Rule — Derivatives of Composite Functions

Single-Variable Chain Rule

When a function is built by composing (nesting) other functions, the chain rule gives us the derivative by multiplying the individual derivatives along the chain.

If $y = f(u)$ and $u = g(x)$, then:

$$dy/dx = (dy/du) \cdot (du/dx) = f'(g(x)) \cdot g'(x)$$

Chain example: $y = \sin(x^2)$

$$u = x^2 \rightarrow du/dx = 2x$$

$$y = \sin(u) \rightarrow dy/du = \cos(u) = \cos(x^2)$$

$$dy/dx = \cos(x^2) \cdot 2x$$

Extended Chain Rule for Longer Chains

If $y = f(g(h(x)))$, then:

$$dy/dx = f'(g(h(x))) \cdot g'(h(x)) \cdot h'(x)$$

More generally: $dy/dx = \prod$ (derivative of each function in the chain)

Multivariable Chain Rule

When multiple variables are involved:

If $z = f(x,y)$ where $x = g(t)$ and $y = h(t)$:

$$dz/dt = (\partial z/\partial x) \cdot (dx/dt) + (\partial z/\partial y) \cdot (dy/dt)$$

This generalises: $dz/dt = \sum_i (\partial z/\partial x_i) \cdot (dx_i/dt)$

6.2 Backpropagation — Chain Rule in Neural Networks

What is Backpropagation?

Backpropagation is the algorithm that computes the gradient of the loss with respect to every weight in a neural network. It uses the chain rule systematically, working backward from the output layer to the input layer.

Forward Pass vs Backward Pass

FORWARD PASS (inference):

$x \rightarrow \text{Layer 1} \rightarrow \text{Layer 2} \rightarrow \dots \rightarrow \text{Output} \rightarrow \text{Loss}$

Purpose: compute predictions and loss

BACKWARD PASS (backpropagation):

$\partial L / \partial w_{\text{last}} \leftarrow \partial L / \partial \text{output} \leftarrow \partial L / \partial \text{Layer2} \leftarrow \partial L / \partial \text{Layer1}$

Purpose: compute gradients using chain rule

Then: update each weight $w \leftarrow w - \alpha \cdot \partial L / \partial w$

Worked Example: 2-Layer Network

Let us trace through backprop for a minimal 2-layer network:

Forward pass:

$$z_1 = w_1 \cdot x + b_1 \quad (\text{linear})$$

$$a_1 = \sigma(z_1) \quad (\text{sigmoid activation})$$

$$z_2 = w_2 \cdot a_1 + b_2 \quad (\text{linear})$$

$$\hat{y} = \sigma(z_2) \quad (\text{output sigmoid})$$

$$L = (y - \hat{y})^2 \quad (\text{MSE loss})$$

Backward pass (chain rule):

$$\partial L / \partial \hat{y} = -2(y - \hat{y})$$

$$\partial \hat{y} / \partial z_2 = \sigma(z_2) \cdot (1 - \sigma(z_2))$$

$$\partial z_2 / \partial w_2 = a_1$$

$$\begin{aligned} \partial L / \partial w_2 &= (\partial L / \partial \hat{y}) \cdot (\partial \hat{y} / \partial z_2) \cdot (\partial z_2 / \partial w_2) \leftarrow \text{chain rule!} \\ &= -2(y - \hat{y}) \cdot \sigma(z_2)(1 - \sigma(z_2)) \cdot a_1 \end{aligned}$$

Continue backward to get $\partial L / \partial w_1$, $\partial L / \partial b_1 \dots$

```
import numpy as np

# Minimal backpropagation example (1 hidden neuron)
```

```

def sigmoid(x):    return 1 / (1 + np.exp(-x))
def sig_d(x):     return sigmoid(x) * (1 - sigmoid(x))

np.random.seed(0)
x = 0.5          # input
y = 1.0          # true label

# Random initialisation
w1, b1, w2, b2 = 0.3, 0.1, -0.5, 0.2

# — FORWARD PASS —————
z1 = w1*x + b1
a1 = sigmoid(z1)
z2 = w2*a1 + b2
y_hat = sigmoid(z2)
loss = (y - y_hat)**2

print(f'Prediction: {y_hat:.4f} | Loss: {loss:.4f}')

# — BACKWARD PASS —————
dL_dyhat = -2*(y - y_hat)
dyhat_dz2 = sig_d(z2)
dz2_dw2 = a1
dz2_da1 = w2
da1_dz1 = sig_d(z1)
dz1_dw1 = x

# Chain rule
dL_dw2 = dL_dyhat * dyhat_dz2 * dz2_dw2
dL_db2 = dL_dyhat * dyhat_dz2
dL_dw1 = dL_dyhat * dyhat_dz2 * dz2_da1 * da1_dz1 * dz1_dw1
dL_db1 = dL_dyhat * dyhat_dz2 * dz2_da1 * da1_dz1

print(f'Gradients: dL/dw2={dL_dw2:.4f} | dL/dw1={dL_dw1:.4f}')

# — UPDATE —————
alpha = 0.1
w1 -= alpha*dL_dw1; b1 -= alpha*dL_db1
w2 -= alpha*dL_dw2; b2 -= alpha*dL_db2
print('Weights updated.')

```

The Vanishing Gradient Problem

In deep networks, the chain rule multiplies many terms together. If each sigmoid derivative is 0.25 (its maximum), and we have 10 layers, the gradient at the first layer is at most $0.25^{10} \approx 0.000001$. The gradient virtually disappears.

Solutions to Vanishing Gradients

1. ReLU Activations: derivative is 1 for positive inputs, not squeezed to ~0.25
2. Batch Normalisation: re-scales activations at each layer to prevent saturation
3. Residual Connections (ResNets): skip connections provide 'gradient highways'

4. LSTM gates: gating mechanism in RNNs preserves gradient flow over time
5. Careful weight initialisation: He, Xavier — prevents gradients from exploding or vanishing initially

Mini Quiz

Q1: In backprop, why do we need to store intermediate activations from the forward pass?

✓ *Answer: The chain rule requires the activation values (a_i , z_i , etc.) to compute the backward-pass multipliers. For example, $\partial L / \partial w_2 = \delta \cdot a_1$ — we need a_1 which was computed in the forward pass.*

Q2: A chain of 20 sigmoid activations, each with derivative 0.2. What is the approximate gradient at the first layer?

✓ *Answer: $0.2^{20} \approx 1 \times 10^{-14}$ — essentially zero! This is the vanishing gradient problem that makes deep sigmoid networks untrainable.*

Chapter 7: Jacobian and Hessian Matrices

7.1 The Jacobian Matrix

From Vectors to Vectors

The gradient handles the case: one output, many inputs (scalar function of a vector). The Jacobian handles: many outputs, many inputs (vector-valued function). It is the matrix of ALL partial derivatives.

Given $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ (n inputs, m outputs)

$$J = \begin{bmatrix} \partial f_1 / \partial x_1 & \partial f_1 / \partial x_2 & \dots & \partial f_1 / \partial x_n \\ \partial f_2 / \partial x_1 & \partial f_2 / \partial x_2 & \dots & \partial f_2 / \partial x_n \\ \dots & \dots & \dots & \dots \\ \partial f_m / \partial x_1 & \partial f_m / \partial x_2 & \dots & \partial f_m / \partial x_n \end{bmatrix}$$

$$J_{ij} = \partial f_i / \partial x_j \quad (\text{row } i, \text{ column } j)$$



Jacobian in Deep Learning

Batch normalisation, softmax, and other multi-output layers require the Jacobian for correct backpropagation.

Softmax: output is a vector of probabilities. Its Jacobian is a matrix relating changes in input logits to changes in output probabilities.

The chain rule in vector form: $\partial L / \partial x = J^T \cdot \partial L / \partial y$

Where J is the Jacobian of the layer and $\partial L / \partial y$ is the incoming gradient vector.

Worked Example: Jacobian of a 2D Function

$$f(x,y) = [x^2y, x+2y]$$

$$J = \begin{bmatrix} \partial f_1 / \partial x & \partial f_1 / \partial y \\ \partial f_2 / \partial x & \partial f_2 / \partial y \end{bmatrix} = \begin{bmatrix} 2xy & x^2 \\ 1 & 2 \end{bmatrix}$$

At point (1,3):

$$J(1,3) = \begin{bmatrix} 2 \cdot 1 \cdot 3 & 1^2 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 6 & 1 \\ 1 & 2 \end{bmatrix}$$

7.2 The Hessian Matrix

Second-Order Derivatives — Curvature

The Hessian matrix contains all second-order partial derivatives of a scalar function. While the gradient tells you the slope, the Hessian tells you the curvature — how quickly the slope itself is changing.

For $f: \mathbb{R}^n \rightarrow \mathbb{R}$, the Hessian H is an $n \times n$ matrix:

$$H = \begin{bmatrix} \partial^2 f / \partial x_1^2 & \partial^2 f / \partial x_1 \partial x_2 & \dots & \partial^2 f / \partial x_1 \partial x_n \\ \partial^2 f / \partial x_2 \partial x_1 & \partial^2 f / \partial x_2^2 & \dots & \partial^2 f / \partial x_2 \partial x_n \\ \dots & \dots & \dots & \dots \end{bmatrix}$$

$$H_{ij} = \partial^2 f / \partial x_i \partial x_j$$

Example: Hessian of $f(x,y) = x^2 + xy + y^2$

First derivatives:

$$\partial f / \partial x = 2x + y \quad \partial f / \partial y = x + 2y$$

Second derivatives:

$$\begin{aligned} \partial^2 f / \partial x^2 &= 2 & \partial^2 f / \partial y^2 &= 2 \\ \partial^2 f / \partial x \partial y &= 1 & \partial^2 f / \partial y \partial x &= 1 \quad (\text{Schwarz: these are equal!}) \end{aligned}$$

$$H = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad (\text{constant — this is a quadratic bowl})$$

Newton's Method — Using the Hessian

Newton's method uses second-order information (the Hessian) to take smarter steps toward the minimum. Instead of just knowing the slope, it knows the curvature — so it can jump directly toward the minimum.

$$\text{Newton update: } w \leftarrow w - H^{-1} \cdot \nabla L$$

$$\text{vs Gradient Descent: } w \leftarrow w - \alpha \cdot \nabla L$$

Newton scales the gradient by the inverse Hessian, adapting the step size to the local curvature.

Method	Uses	Pros	Cons
Gradient Descent	∇L only (first order)	Simple, scalable to millions of params	Slow convergence, sensitive to α
Newton's Method	$\nabla L + H$ (second order)	Fast convergence (quadratic)	H is $n \times n$ — infeasible for $n = 10^6$
Quasi-Newton (L-BFGS)	Approximates H^{-1}	Fast for medium-scale problems	Memory intensive
Adam Optimizer	Adaptive per-param step	Practical, widely used in DL	More hyperparameters

```
import numpy as np

# Newton's method for minimising f(x) = x^4 - 3x^3 + 2
def f(x):    return x**4 - 3*x**3 + 2
def df(x):   return 4*x**3 - 9*x**2      # first derivative
def d2f(x):  return 12*x**2 - 18*x       # second derivative (Hessian in 1D)

x = 3.0      # starting point
print(f'Start: x={x:.4f}, f(x)={f(x):.4f}')

for i in range(10):
    grad = df(x)
    curv = d2f(x)
    if abs(curv) < 1e-10: break
    x = x - grad / curv      # Newton step
    print(f'Step {i+1}: x={x:.6f}, f(x)={f(x):.6f}, f\'={grad:.4f}')
```

Mini Quiz

Q1: A Jacobian is (4×3). What does this tell us about the function?

✓ **Answer:** The function maps 3 inputs to 4 outputs: $f: \mathbb{R}^3 \rightarrow \mathbb{R}^4$. There are $4 \times 3 = 12$ partial derivatives in total.

Q2: Why is Newton's method usually not used directly for training deep neural networks?

✓ **Answer:** The Hessian is $n \times n$ where n is the number of weights. For a network with 10 million weights, the Hessian has 10^{14} entries — impossible to store or invert. Gradient descent is $O(n)$ per step; Newton is $O(n^3)$.

Chapter 8: Advanced Optimisation Techniques

8.1 Momentum

Why Standard GD Oscillates

Standard gradient descent can oscillate in narrow valleys — taking large steps sideways while making slow progress toward the minimum. Momentum addresses this by accumulating velocity in consistent directions.

Momentum update (β typically = 0.9):

$$v \leftarrow \beta \cdot v + (1-\beta) \cdot \nabla L \quad (\text{velocity — exponential moving average of gradients})$$

$$w \leftarrow w - \alpha \cdot v$$

The velocity v builds up in directions where gradients consistently point one way, and cancels out in directions where gradients oscillate.

8.2 RMSprop and Adam

Adaptive learning rate methods automatically adjust the learning rate per parameter, making training much more robust.

Adam Optimizer (Adaptive Moment Estimation)

Adam combines momentum (first moment) with RMSprop (second moment):

$$m \leftarrow \beta_1 \cdot m + (1-\beta_1) \cdot \nabla L \quad (\text{first moment: mean of gradient})$$

$$v \leftarrow \beta_2 \cdot v + (1-\beta_2) \cdot (\nabla L)^2 \quad (\text{second moment: mean of squared gradient})$$

Bias correction:

$$\hat{m} = m / (1-\beta_1^t) \quad (\text{corrects for initialisation at zero})$$

$$\hat{v} = v / (1-\beta_2^t)$$

$$\text{Update: } w \leftarrow w - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

Typical values: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$, $\alpha=0.001$

8.3 Learning Rate Schedules

Instead of a fixed learning rate, we can reduce it over time (learning rate decay). A large initial rate enables fast progress; a smaller later rate allows fine-tuning near the minimum.

Schedule	Formula	Effect
Step Decay	$\alpha = \alpha_0 \times \gamma^{(\text{epoch}/\text{drop})}$	Drops by factor γ every few epochs
Exponential	$\alpha = \alpha_0 \times e^{(-\lambda t)}$	Smooth continuous decrease
Cosine Annealing	$\alpha = \alpha_{\max} \cdot \cos(\pi t/T)/2$	Gentle decay with warm restarts possible
1-Cycle Policy	Increases then decreases	Often best in practice (fast.ai)

```
import numpy as np

# Adam optimizer implemented from scratch
def adam(grad_fn, w_init, alpha=0.001, beta1=0.9, beta2=0.999,
         eps=1e-8, n_steps=200):
    w = w_init.copy()
    m = np.zeros_like(w) # first moment
    v = np.zeros_like(w) # second moment

    history = []
    for t in range(1, n_steps+1):
        g = grad_fn(w) # get gradient
        m = beta1*m + (1-beta1)*g # update first moment
        v = beta2*v + (1-beta2)*g**2 # update second moment
        m_hat = m / (1 - beta1**t) # bias correction
        v_hat = v / (1 - beta2**t)
        w -= alpha * m_hat / (np.sqrt(v_hat) + eps)
        history.append(w.copy())
    return w, history

# Minimise f(w) = w1^2 + 10*w2^2 (elongated bowl - hard for plain GD)
def grad_fn(w): return np.array([2*w[0], 20*w[1]])

w_start = np.array([5.0, 5.0])
w_opt, hist = adam(grad_fn, w_start)
print('Adam result:', np.round(w_opt, 4)) # Should be near [0, 0]
```

Mini Quiz

Q1: What problem does momentum solve that plain gradient descent has?

✓ *Answer: Oscillation in narrow valleys. Momentum accumulates velocity in consistent directions, speeding up progress in the correct direction while damping oscillations perpendicular to it.*

Q2: Adam uses two exponential moving averages (m and v). What does each represent?

✓ *Answer: m = first moment = moving average of gradients (direction). v = second moment = moving average of squared gradients (magnitude/variance). Together they enable adaptive, normalised updates.*

Chapter 9: Neural Networks — The Full Calculus

9.1 A Neural Network is a Composite Function

A deep neural network with L layers computes a composition of functions. Each layer is a linear transformation followed by an activation function:

Layer l : $a_l = f(W_l \cdot a_{l-1} + b_l)$ where f is an activation function

Full network: $\hat{y} = f_L(W_L \cdot f_{L-1}(W_{L-1} \cdot \dots f_1(W_1 x + b_1) \dots + b_{L-1}) + b_L)$

This is a big composite function of ALL weights in ALL layers.

9.2 Full Backpropagation Algorithm

The backprop algorithm efficiently computes $\partial L / \partial W_l$ for every layer l using the chain rule, propagating error signals backward:

Define error signal at layer l : $\delta_l = \partial L / \partial z_l$

Output layer (L): $\delta_L = \partial L / \partial a_L \odot f'(z_L)$ ($\odot$ = element-wise multiply)

Hidden layer ($l < L$): $\delta_l = (W_{l+1}^T \cdot \delta_{l+1}) \odot f'(z_l)$

Weight gradients: $\partial L / \partial W_l = \delta_l \cdot a_{l-1}^T$ (outer product)

Bias gradients: $\partial L / \partial b_l = \delta_l$

Update: $W_l \leftarrow W_l - \alpha \cdot \partial L / \partial W_l$

Why This is Efficient

Naive approach: compute $\partial L / \partial w$ for every weight independently $\rightarrow O(n^2)$ operations

Backpropagation: reuses intermediate δ_l values, passing them backward $\rightarrow O(n)$ operations

This is why training a 100-million-parameter transformer is feasible in hours on a GPU, but would take years with a naive approach.

9.3 Batch Operations — Matrix Form of Backprop

In practice, we process batches of B samples simultaneously. All the scalar operations become matrix operations:

$$\text{Forward: } Z_l = W_l \cdot A_{l-1} + b_l \quad \text{shapes: } (n_l \times B) = (n_l \times n_{l-1}) \cdot (n_{l-1} \times B) + (n_l \times 1)$$

$$A_l = f(Z_l) \quad \text{element-wise activation}$$

$$\text{Backward: } \Delta_l = (W_{l+1}^T \cdot \Delta_{l+1}) \odot f'(Z_l) \quad \text{shape: } (n_l \times B)$$

$$\text{Gradient: } \partial L / \partial W_l = (1/B) \cdot \Delta_l \cdot A_{l-1}^T \quad \text{average over batch}$$

```
import numpy as np

# Full backprop for a 2-layer network (batch version)
def relu(z):    return np.maximum(0, z)
def relu_d(z):  return (z > 0).astype(float)
def sigmoid(z): return 1/(1+np.exp(-z))
def sig_d(z):   return sigmoid(z)*(1-sigmoid(z))

np.random.seed(42)
# Architecture: 2 inputs → 4 hidden → 1 output
n0, n1, n2 = 2, 4, 1
B = 8      # batch size

# Random weights (Xavier init)
W1 = np.random.randn(n1,n0) * np.sqrt(2/n0)
b1 = np.zeros((n1,1))
W2 = np.random.randn(n2,n1) * np.sqrt(2/n1)
b2 = np.zeros((n2,1))

# Random data
X = np.random.randn(n0, B)
y = (np.random.rand(1, B) > 0.5).astype(float)

alpha = 0.05

for epoch in range(100):
    # ——— FORWARD PASS ———
    Z1 = W1 @ X + b1          # (4, 8)
    A1 = relu(Z1)             # (4, 8)
    Z2 = W2 @ A1 + b2         # (1, 8)
    A2 = sigmoid(Z2)          # (1, 8) — predictions

    loss = -np.mean(y*np.log(A2+1e-8) + (1-y)*np.log(1-A2+1e-8))

    # ——— BACKWARD PASS ———
    dA2 = -(y/A2 - (1-y)/(1-A2)) / B
    dZ2 = dA2 * sig_d(Z2)
    dW2 = dZ2 @ A1.T
    db2 = dZ2.sum(axis=1, keepdims=True)
```

```

dA1 = W2.T @ dZ2
dZ1 = dA1 * relu_d(Z1)
dW1 = dZ1 @ X.T
db1 = dZ1.sum(axis=1, keepdims=True)

# ----- UPDATE -----
W1 -= alpha*dW1; b1 -= alpha*db1
W2 -= alpha*dW2; b2 -= alpha*db2

if epoch % 20 == 0: print(f'Epoch {epoch:3d} | Loss: {loss:.4f}')

```

Mini Quiz

Q1: In backprop, $\delta_l = (W_{l+1}^T \cdot \delta_{l+1}) \odot f'(z_l)$. Why do we use W_{l+1}^T (transpose)?

✓ *Answer: We are flowing gradients BACKWARD through the layer, which reverses the direction. The transpose reverses the linear transformation mathematically, routing each error signal to the correct source neuron.*

Q2: Why is the weight gradient $\partial L / \partial W_l$ divided by batch size B ?

✓ *Answer: We average the gradient over all B samples in the batch. Without dividing by B , the gradient magnitude would grow with batch size, requiring us to change the learning rate for every different batch size.*

Chapter 10: Logistic Regression — Calculus in Action

10.1 The Model

Logistic regression predicts a probability that an input belongs to class 1. It combines a linear function with the sigmoid activation:

$$\hat{y} = \sigma(w^T x + b) = 1 / (1 + e^{-(w^T x + b)})$$

Output: $\hat{y} \in (0,1)$ — a probability

10.2 Binary Cross-Entropy Loss

$$L(w,b) = -(1/n) \sum_i [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

When $y_i=1$: want $\hat{y}_i \rightarrow 1 \rightarrow -\log(\hat{y}_i) \rightarrow 0$ (small loss)

When $y_i=0$: want $\hat{y}_i \rightarrow 0 \rightarrow -\log(1-\hat{y}_i) \rightarrow 0$ (small loss)

10.3 Gradient Derivation — Step by Step

Let us derive the gradient $\partial L / \partial w$ using the chain rule:

$$\text{Let: } z = w^T x + b, \quad \hat{y} = \sigma(z), \quad L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

$$\partial L / \partial \hat{y} = -y / \hat{y} + (1-y) / (1-\hat{y})$$

$$\partial \hat{y} / \partial z = \sigma(z) \cdot (1 - \sigma(z)) = \hat{y} \cdot (1 - \hat{y})$$

$$\begin{aligned} \partial L / \partial z &= \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial z \\ &= [-y / \hat{y} + (1-y) / (1-\hat{y})] \cdot \hat{y} (1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} \\ &= \hat{y} - y \quad \leftarrow \text{beautifully simple!} \end{aligned}$$

$$\begin{aligned} \partial L / \partial w &= (\partial L / \partial z) \cdot (\partial z / \partial w) = (\hat{y} - y) \cdot x \\ \partial L / \partial b &= (\hat{y} - y) \end{aligned}$$

```
import numpy as np
from sklearn.datasets import make_classification
```

```

# Generate binary classification data
X, y = make_classification(n_samples=200, n_features=2,
                          n_redundant=0, random_state=42)
y = y.reshape(-1,1)

# Logistic regression from scratch
def sigmoid(z): return 1/(1+np.exp(-z))

w = np.zeros((2,1))
b = 0.0
alpha = 0.1

for epoch in range(1000):
    n = len(X)
    z = X @ w + b          # linear combination
    y_hat = sigmoid(z)      # apply sigmoid
    loss = -np.mean(y*np.log(y_hat+1e-8) + (1-y)*np.log(1-y_hat+1e-8))

    # Gradients (from our derivation above)
    dL_dw = (1/n) * X.T @ (y_hat - y)
    dL_db = (1/n) * np.sum(y_hat - y)

    w -= alpha * dL_dw
    b -= alpha * dL_db

    if epoch % 200 == 0:
        acc = np.mean((y_hat > 0.5) == y)
        print(f'Epoch {epoch:4d} | Loss: {loss:.4f} | Accuracy: {acc:.3f}')

```

Mini Quiz

Q1: Why is cross-entropy preferred over MSE loss for classification?

✓ *Answer: (1) MSE gives very small gradients when predictions are confidently wrong (e.g. $\hat{y}=0.99$, $y=0$), due to the small sigmoid derivative. Cross-entropy gradient is simply $(\hat{y}-y)$, which is large when wrong. (2) Cross-entropy has a probabilistic interpretation (maximum likelihood estimation).*

Q2: The gradient of cross-entropy w.r.t. weights is $(1/n)X^T(\hat{y}-y)$. What does $(\hat{y}-y)$ represent?

✓ *Answer: The prediction error — how far each prediction is from the true label. It is the 'signal' that tells the model in which direction to change its weights. When predictions are perfect, error is 0 and no update occurs.*

Chapter 11: Higher-Order Derivatives and Taylor Series

11.1 Higher-Order Derivatives

We can take the derivative of a derivative. The second derivative $f''(x) = d^2f/dx^2$ tells us about the rate of change of the rate of change — in other words, the curvature of the function.

Derivative Order	Notation	Physical Meaning	ML Meaning
1st: $f'(x)$	df/dx	Velocity (rate of change)	Gradient — direction of steepest change
2nd: $f''(x)$	d^2f/dx^2	Acceleration (curvature)	Hessian diagonal — convexity/concavity
3rd: $f'''(x)$	d^3f/dx^3	Jerk (rate of change of accel.)	Rarely used in practice
nth: $f^{(n)}(x)$	d^nf/dx^n	Higher-order behaviour	Taylor expansion coefficients

Sign of Second Derivative → Convexity

$f''(x) > 0$ everywhere → function is CONVEX (bowl shape)
→ gradient descent finds the GLOBAL minimum

$f''(x) < 0$ everywhere → function is CONCAVE (hill shape)

Mixed signs → non-convex (has local minima and maxima)
→ gradient descent may get stuck

Convexity in ML

Convex problems (like logistic regression, SVM, PCA) are 'easy' — gradient descent always finds the global minimum.

Non-convex problems (like deep neural networks) have many local minima and saddle points. Gradient descent may not find the global minimum — but in practice, it finds good solutions anyway.

Understanding convexity helps you understand when to trust your model's convergence.

11.2 Taylor Series — Approximating Functions Locally

What is a Taylor Series?

A Taylor series expresses any smooth function as an infinite sum of polynomial terms. Near a point $x = a$, we can approximate $f(x)$ using its derivatives at that point.

Taylor series of $f(x)$ around point a :

$$f(x) \approx f(a) + f'(a)(x-a) + f''(a)(x-a)^2/2! + f'''(a)(x-a)^3/3! + \dots$$

$$\sum_{n=0}^{\infty} f^{(n)}(a) \cdot (x-a)^n / n!$$

When $a=0$, this is called a Maclaurin series.

Important Taylor Expansions

$$e^x = 1 + x + x^2/2! + x^3/3! + \dots \quad (\text{all terms positive})$$

$$\ln(1+x) = x - x^2/2 + x^3/3 - x^4/4 + \dots \quad (\text{for } |x| < 1)$$

$$\sin(x) = x - x^3/3! + x^5/5! - \dots \quad (\text{odd powers})$$

$$\cos(x) = 1 - x^2/2! + x^4/4! - \dots \quad (\text{even powers})$$

$$\sigma(x) = 0.5 + x/4 - x^3/48 + \dots \quad (\text{sigmoid near } x=0)$$

Taylor Series in Machine Learning

Second-order Taylor expansion of loss $L(w)$ around current weights w_0 :

$$L(w) \approx L(w_0) + \nabla L \cdot (w - w_0) + \frac{1}{2}(w - w_0)^T H (w - w_0)$$

This quadratic approximation is the basis for:

- Newton's method (minimise the approximation exactly)
- Trust region methods (stay in region where approximation is valid)
- Understanding why gradient descent works: the first-order term tells you which direction to step.

```
import numpy as np

# Taylor approximation of exp(x) around x=0
x = np.array([0.5, 1.0, 1.5, 2.0])
```



```

print(f'{'x':>6} {'True e*':>10} {'Order 1':>10} {'Order 3':>10} {'Order
5':>10}')
print('-'*55)
for xi in x:
    true_val = np.exp(xi)
    order1 = 1 + xi
    order3 = 1 + xi + xi**2/2 + xi**3/6
    order5 = order3 + xi**4/24 + xi**5/120
    print(f'{'xi':>6.1f} {'true_val':>10.5f} {'order1':>10.5f} {'order3':>10.5f}
{'order5':>10.5f}')
```

Mini Quiz

Q1: What does $f''(x) > 0$ tell you about the shape of the function and its implications for optimization?

✓ *Answer: The function is convex — it curves upward like a bowl. Gradient descent is guaranteed to find the global minimum (there are no local minima to get stuck in).*

Q2: Why is the Taylor series useful in understanding gradient descent updates?

✓ *Answer: The first-order Taylor expansion $L(w+\Delta w) \approx L(w) + \nabla L \cdot \Delta w$ shows that moving in the direction $\Delta w = -\alpha \nabla L$ decreases L by approximately $\alpha \| \nabla L \|^2$. This justifies gradient descent as a first-order optimization method.*

Chapter 12: Numerical Differentiation and Automatic Differentiation

12.1 Numerical Differentiation

Finite Differences

When a function is too complex to differentiate analytically, or when only its values are known, we can approximate derivatives numerically using finite differences.

Forward difference: $f'(x) \approx [f(x+h) - f(x)] / h$ ($O(h)$ error)
 Backward difference: $f'(x) \approx [f(x) - f(x-h)] / h$ ($O(h)$ error)
 Central difference: $f'(x) \approx [f(x+h) - f(x-h)] / (2h)$ ($O(h^2)$ error — much better!)

Typical choice: $h = 1e-5$

```
import numpy as np

def numerical_gradient(f, x, h=1e-5):
    '''Compute gradient of f at x using central differences'''
    grad = np.zeros_like(x, dtype=float)
    for i in range(len(x)):
        x_plus = x.copy(); x_plus[i] += h
        x_minus = x.copy(); x_minus[i] -= h
        grad[i] = (f(x_plus) - f(x_minus)) / (2*h)
    return grad

# Test: gradient of f(x) = x1^2 + 3x1x2 + x2^3
def f(x): return x[0]**2 + 3*x[0]*x[1] + x[1]**3

x = np.array([2.0, 1.0])
num_grad = numerical_gradient(f, x)
anal_grad = np.array([2*x[0]+3*x[1], 3*x[0]+3*x[1]**2])

print('Numerical gradient: ', np.round(num_grad, 6))
print('Analytical gradient: ', np.round(anal_grad, 6))
print('Max difference:      ', np.max(np.abs(num_grad - anal_grad)))
```

12.2 Automatic Differentiation (Autograd)

What is Autograd?

Modern deep learning frameworks (PyTorch, TensorFlow, JAX) use automatic differentiation — a technique that computes exact derivatives (not approximations) by tracking operations in a computational graph and applying the chain rule automatically.

Method	Type	Accuracy	Used In
Symbolic diff	Manual algebra	Exact	Math textbooks, CAS
Numerical diff	Finite differences	Approximate	Gradient checking
Forward-mode AD	Dual numbers	Exact	Small input, many outputs
Reverse-mode AD	Backprop = reverse	Exact	Neural networks (many params, 1 loss)

```
# PyTorch autograd example
import torch

# Create tensors with gradient tracking
x = torch.tensor([2.0, 1.0], requires_grad=True)

# Define computation (same function as above)
f = x[0]**2 + 3*x[0]*x[1] + x[1]**3

# Compute gradient automatically
f.backward()      # reverse-mode autodiff

print('f value:', f.item())      # 4 + 6 + 1 = 11
print('Gradient:', x.grad)      # Should be [7., 9.]

# — NumPy vs PyTorch comparison —
import numpy as np
xn = np.array([2.0, 1.0])
def fn(x): return x[0]**2 + 3*x[0]*x[1] + x[1]**3
def grad_fn(x): return np.array([2*x[0]+3*x[1], 3*x[0]+3*x[1]**2])

print('NumPy analytical:', grad_fn(xn))      # [7. 9.]
print('Autograd:      ', x.grad.numpy())     # [7. 9.]
```

Why Reverse-Mode is Used for Neural Networks

A neural network has 1 loss output and millions of weight inputs.

Forward-mode AD computes the gradient of ONE output w.r.t. ALL inputs simultaneously — efficient when there are few inputs.

Reverse-mode AD (backpropagation) computes the gradient of ONE output w.r.t. ALL inputs in ONE backward pass — $O(n)$ regardless of how many inputs there are.

For $n=10^7$ weights and 1 loss: reverse-mode is $10^7\times$ more efficient than forward-mode.

Mini Quiz

Q1: Central difference is more accurate than forward difference. Why?

✓ Answer: Central difference has $O(h^2)$ error while forward difference has $O(h)$ error. With $h=1e-5$, central difference error is $\sim 10^{-10}$ while forward difference error is $\sim 10^{-5}$. The central method is symmetric and cancels out first-order error terms.

Q2: What is the computational complexity advantage of reverse-mode AD for neural network training?

✓ Answer: It computes gradients of ONE scalar (the loss) w.r.t. ALL parameters in ONE backward pass — $O(n)$ operations. Forward-mode would need n separate passes (one per parameter), costing $O(n^2)$.

Chapter 13: Calculus in Probability and Statistics

13.1 Probability Density Functions

A probability density function (PDF) $f(x)$ describes the likelihood of a continuous random variable taking a value near x . Crucially, it is computed using integration (the other branch of calculus):

$$P(a \leq X \leq b) = \int_a^b f(x) dx \quad (\text{area under the PDF curve})$$

$$\text{Requirement: } \int_{-\infty}^{\infty} f(x) dx = 1 \quad (\text{total probability} = 1)$$

Normal (Gaussian) PDF:

$$f(x) = (1/\sqrt{2\pi\sigma^2}) \cdot \exp(-(x-\mu)^2/(2\sigma^2))$$

13.2 Maximum Likelihood Estimation

Maximum likelihood estimation (MLE) finds the model parameters that make the observed data most probable. This is a calculus optimization problem: find parameters that maximise the log-likelihood.

$$\text{Likelihood: } L(\theta) = \prod_i p(x_i|\theta)$$

$$\text{Log-likelihood: } \ell(\theta) = \sum_i \log p(x_i|\theta) \quad (\text{log turns product into sum})$$

$$\text{MLE: } \theta^* = \underset{\theta}{\operatorname{argmax}} \ell(\theta)$$

$$\text{Set } \partial \ell / \partial \theta = 0 \text{ and solve}$$

Cross-entropy loss IS negative log-likelihood:
Training with cross-entropy = MLE for classification models

Deep Connection: Loss Functions = Negative Log-Likelihood

MSE (Mean Squared Error) = Assumes Gaussian noise $\rightarrow$ MLE under Gaussian model

Cross-Entropy = Negative log-likelihood for Bernoulli (binary) or Categorical model

KL Divergence = Difference in log-likelihoods between two distributions

When you train a neural network minimising cross-entropy, you are doing maximum likelihood estimation of the model parameters. The calculus of probability and the calculus of optimization are the SAME thing.

13.3 Gradient of the KL Divergence

KL divergence measures how different one probability distribution is from another. It appears in variational autoencoders (VAEs), information theory, and many modern ML algorithms.

$$\begin{aligned} \text{KL}(P||Q) &= \sum_x P(x) \cdot \log[P(x)/Q(x)] \quad (\text{discrete}) \\ &= \int p(x) \cdot \log[p(x)/q(x)] dx \quad (\text{continuous}) \end{aligned}$$

For Gaussian distributions $N(\mu_1, \sigma_1^2)$ vs $N(0,1)$:

$$\text{KL} = \frac{1}{2} [\sigma_1^2 + \mu_1^2 - 1 - \log(\sigma_1^2)]$$

$$\partial \text{KL} / \partial \mu_1 = \mu_1 \quad \leftarrow \text{simple gradient!}$$

$$\partial \text{KL} / \partial \sigma_1 = \sigma_1 - 1/\sigma_1$$

Mini Quiz

Q1: If we take the log of the likelihood function, does the argmax change?

 *Answer: No! log is a monotonically increasing function — it preserves the location of the maximum. But it turns products into sums, making the calculus much simpler.*

Q2: Training with MSE loss implicitly assumes what about the noise in the data?

 *Answer: It assumes Gaussian (normally distributed) noise around the true values. Minimising MSE is equivalent to Maximum Likelihood Estimation assuming $y = f(x) + \epsilon$ where $\epsilon \sim N(0, \sigma^2)$.*

Chapter 14: Summary and Further Reading

14.1 Complete Concept Map

Calculus Concept	Formula	ML Application
Derivative	$f'(x) = \lim[f(x+h)-f(x)]/h$	Gradient, weight updates
Chain Rule	$d/dx f(g(x)) = f'(g) \cdot g'$	Backpropagation
Partial Derivative	$\partial f / \partial x_i$	Gradient components
Gradient	$\nabla f = [\partial f / \partial x_1, \dots, \partial f / \partial x_n]$	Direction of steepest ascent
Gradient Descent	$w \leftarrow w - \alpha \cdot \nabla L$	All model training
Jacobian	$J_{ij} = \partial f_i / \partial x_j$	Softmax, batch norm backprop
Hessian	$H_{ij} = \partial^2 f / \partial x_i \partial x_j$	Curvature, Newton's method
Taylor Expansion	$f(x) \approx f(a) + f'(a)(x-a) + \dots$	Optimization analysis
MSE Gradient	$dL/dw = -(2/n) \sum x(y-\hat{y})$	Linear regression training
CE Gradient	$dL/dw = (1/n) X^T (\hat{y} - y)$	Logistic regression, softmax
Sigmoid Derivative	$\sigma'(x) = \sigma(x)(1-\sigma(x))$	Vanishing gradient analysis
Convexity	$f''(x) > 0$	Global convergence guarantee

14.2 The Big Picture in 5 Steps

1. You have DATA: inputs X and targets y .
2. You define a MODEL: $\hat{y} = f(X; w)$ where w are learnable parameters.
3. You define a LOSS: $L(w)$ measures how wrong the model is.
4. You compute GRADIENTS: ∇L tells you which direction to change w .
5. You run GRADIENT DESCENT: iterate $w \leftarrow w - \alpha \cdot \nabla L$ until L is small.

Calculus is what makes steps 4 and 5 possible. Without derivatives, you could not compute gradients. Without gradients, you could not train any model.

The Chain Rule = Backpropagation = All of Deep Learning Training

Everything in this book ultimately reduces to one idea: the chain rule.

A neural network is a composite function. Training it requires the gradient of the loss with respect to every weight. The chain rule lets us compute all these gradients efficiently by one backward pass through the network.

Understand the chain rule deeply, and you understand how every neural network in the world is trained.

14.3 Recommended Next Steps

Mathematics

- Linear Algebra (companion volume) — vectors, matrices, eigenvalues used alongside calculus
- Probability & Statistics — distributions, Bayes' theorem, statistical inference
- Convex Optimization — Boyd & Vandenberghe (free online) for deeper optimization theory

Books

- 'Mathematics for Machine Learning' by Deisenroth, Faisal & Ong — free PDF, covers both calculus and linear algebra
- 'Deep Learning' by Goodfellow, Bengio & Courville — Chapters 3-6 cover relevant math deeply
- 'Calculus' by James Stewart — comprehensive single-variable reference
- 'Pattern Recognition and Machine Learning' by Bishop — probabilistic ML with full math

Practical Projects

6. Implement gradient descent from scratch for linear and logistic regression (no sklearn)
7. Implement a 3-layer neural network with full backpropagation from scratch using only NumPy
8. Implement the Adam optimiser from scratch and compare with standard GD on a non-convex function
9. Use PyTorch autograd to compute gradients of complex functions and verify against numerical differentiation
10. Plot the loss landscape of a simple neural network and observe local minima and saddle points

14.4 Final Comprehensive Quiz

Mini Quiz

Q1: What does the chain rule say, and why is it the foundation of backpropagation?

✓ Answer: For a composite function $y=f(g(x))$: $dy/dx = f'(g(x)) \cdot g'(x)$. Backpropagation applies the chain rule recursively through all layers of a neural network to compute $\partial L/\partial w$ for every weight in one backward pass.

Q2: A model trained with MSE loss has stopped improving. The training loss is flat. What are the three most likely explanations?

✓ Answer: (1) Learning rate is too small — gradients are too tiny. (2) Gradient has vanished — activations are saturated. (3) The model has converged to a local minimum or saddle point.

Q3: What is the difference between the gradient and the Jacobian?

✓ Answer: Gradient: maps a vector input to a scalar output — result is a vector of partial derivatives. Jacobian: maps a vector input to a vector output — result is a matrix of all partial derivatives ($m \times n$ matrix for $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$).

Q4: Why is Adam usually better than plain gradient descent for training neural networks?

✓ Answer: Adam maintains per-parameter adaptive learning rates using first and second moment estimates. It handles sparse gradients well, scales well for large models, and is robust to hyperparameter choice. Plain GD uses the same learning rate for all parameters regardless of their gradient history.

Q5: Explain in plain language why the vanishing gradient problem occurs in deep sigmoid networks.

✓ Answer: The sigmoid derivative is at most 0.25. In backpropagation, this factor gets multiplied for each layer. After 10 layers: $0.25^{10} \approx 10^{-6}$. The gradient at the first layer is essentially zero — weights there cannot learn.

End of Calculus for Machine Learning, Deep Learning, AI & Data Science

Master the derivative. Master machine learning.